

データの頻度集計

医療データ科学実習

(Practice of Biomedical Data Science)

第5回 2026年5月19日

医療データ科学 臨床統計学

matsui.k@med.nagoya-u.ac.jp

emoto.ryo.5m@kyoto-u.ac.jp

前回のSlidoの質問に対する回答

Q. Excelでデータ収集してもそれをcsvに変換しRで解析したほうが良いとのことですが、Excelでのデータ収集時、データの型に関して気を付けなければいけない点を教えてください。

A. カテゴリ型の変数に関しては、不要なスペースの混在などがあるとRへ読み込んだ後のFactor型への変換時に別のカテゴリとして扱われてしまうため、このような表記ゆれについては注意が必要です。エクセルの機能のデータタブにある【入力規則】などはこの対策として利用可能と思います。

数値型の変数の欠測を文字で表すことなどもその列全体が文字型として読み込まれ余計な前処理が必要になるため、注意が必要です(入力なしのセルとしておくとな多くの場合NAで読み込まれます)。

セルの値がエクセル内の関数で計算されている場合なども、エラー値が読み込み時に想定外の型になる可能性があるため、目視で確認できないデータが大きい場合は特に注意が必要です(データタブにある【フィルター】機能で列にある全てのパターンを確認可能です)。

Slidoで質疑応答に参加しよう

医療データ科学実習第2回

Q&A

Slido

医療データ科学実習第2回
2025/04/15~2025/04/22
#2974 065

ライブインタラクシ...

Slido を切り替え

ダークモード

Slido について

Slido を無料で試す

質問を入力

人気 最近

質問投稿部分

1 個の質問

匿名
6 日前

テスト質問です

Moderator
6 日前

テスト質問了解です

人の質問に投票できる
人気の質問は上位に表示される

参加
URL: <https://app.sli.do/event/nx4nDW5x9rkD9RJrEeLvC3>

- 匿名で質疑応答に参加できるプラットフォーム
- スマホからでも簡単に利用できます
- 講義後1週間解放しておくので自由に質問やコメントを投稿してください

主催者としてログイン -
プレゼンテーション モード
許可可能な使用 - Slido のプライバシー
Cookie の設定
© 2012-2025 Slido - 67.29.3

slido



Chap1. Rの組み込み関数を用いた データの集計

- 1次元の離散変数の頻度集計, 連続変数の頻度集計
- 2次元の離散変数のクロス集計
- 離散変数のクロス集計における要約統計量算出

使用データセット: smoking_tbl_df

英国の健康調査に基づく、喫煙に関するデータ

- 対象: イギリスにおける成人1691名の調査データ
- 変数一覧:
 - gender: 性別 (male, female)
 - age: 年齢 (整数)
 - marital_status: 婚姻状況 (single, married など)
 - highest_qualification: 最終学歴 / nationality: 国籍
 - ethnicity: 民族的背景 / gross_income: 所得水準
 - region: 地域 / smoke: 喫煙の有無 (yes, no)
 - amt_weekends: 週末の喫煙本数 (整数)
 - amt_weekdays: 平日の喫煙本数 (整数)
 - type: 喫煙者における喫煙スタイル

データセットの読み込みと確認

演習: ライブラリMedDataSetsのインストール、読み込みを行い
データセットsmoking_tbl_dfの内容を確認してください

data() 関数で利用可能なデータセットをロードできます

```
library(MedDataSets)
data("smoking_tbl_df")

# データの形式を確認
class(smoking_tbl_df)  # tbl: パッケージ固有の属性
# tibble形式を通常data.frameに変換
smoking_tbl_df <- as.data.frame(smoking_tbl_df)

# データの内容を確認
head(smoking_tbl_df)
summary(smoking_tbl_df)
```

カテゴリ変数の度数

- ・`table()`: 各カテゴリの出現回数(度数)を集計

入力オブジェクトは数値であっても文字型であっても
Factor型として処理される

```
> freq_tab <- table(smoking_tbl_df$gender)
> freq_tab
```

Female	Male
965	726

カテゴリ変数の相対度数

- ・`prop.table()`: 度数表に基づき、全体に対する割合を計算

相対度数は合計が 1 になるように計算される

```
> prop.table(freq_tab)

      Female      Male 
0.5706682 0.4293318 

> prop.table(freq_tab)*100  # パーセント表示 (割合 × 100)

      Female      Male 
57.06682 42.93318
```


table() 関数におけるNAの扱い

useNA 引数で NA の表示有無を制御

- "no": デフォルト (NA除外)
- "ifany": NAがある場合のみ表示
- "always": 常に表示 (0も出力)

```
> x <- c("A", "B", NA, "A", "C", NA)
```

```
> table(x, useNA = "ifany")
```

x

A	B	C <NA>
2	1	1 2

```
> table(x, useNA = "always")
```

x

A	B	C <NA>
2	1	1 2

prop.table() 関数におけるNAの扱い

元になる table オブジェクトの情報を引き継いで計算される

```
> # NAを除外した相対度数
> freq_tab = table(x, useNA = "no")
> prop.table(freq_tab)
x
      A      B      C
0.50 0.25 0.25
>
> # NAを考慮した相対度数
> freq_tab = table(x, useNA = "always")
> prop.table(freq_tab)
x
      A      B      C      <NA>
0.3333333 0.1666667 0.1666667 0.3333333
```

NAを除外

連続変数の階級分けと頻度集計

`cut()` 関数: 連続値を区間に分け、factor型に変換

- `breaks` 引数で区間の区切り位置を指定する

```
> # 年齢を区間(0,20], (20,40], (40,60], (60,∞)に分割
> age_category <- cut(smoking_tbl_df$age, breaks = c(0, 20, 40, 60, Inf))
> # 分類結果を確認
> head(age_category)
[1] (20,40] (40,60] (20,40] (20,40] (20,40] (20,40]
Levels: (0,20] (20,40] (40,60] (60,Inf]
> # 度数を集計
> table(age_category)
age_category
(0,20] (20,40] (40,60] (60,Inf]
    73    558    519    541
> # 相対度数を計算
> prop.table(table(age_category))
age_category
(0,20] (20,40] (40,60] (60,Inf]
0.04316972 0.32998226 0.30691898 0.31992904
```

二変数のクロス集計

`table()` の引数に2つのベクトルを指定することでクロス集計が可能

```
> tab2 <- table(smoking_tbl_df$gender, smoking_tbl_df$smoke)
> tab2
```

	No	Yes
Female	731	234
Male	539	187

二変数のクロス集計と割合

`prop.table()` 関数により全体に対する割合を算出できる

- `margin` を指定することで行方向・列方向の割合を計算

```
> prop.table(tab2) # 全体の割合
```

	No	Yes
Female	0.4322886	0.1383797
Male	0.3187463	0.1105855

```
> prop.table(tab2, margin=1) # 行ごとの割合
```

	No	Yes
Female	0.7575130	0.2424870
Male	0.7424242	0.2575758

```
> prop.table(tab2, margin=2) # 列ごとの割合
```

	No	Yes
Female	0.5755906	0.5558195
Male	0.4244094	0.4441805

演習問題1

1. `amt_weekends` を「20本以下」と「20本より多い」に階級分けしてください
2. 階級分けした `amt_weekends` の頻度と相対度数をNAを考慮して求めてください
3. 階級分けした `amt_weekends` と `smoke` のクロス集計表を作成し、`amt_weekends` の欠損の理由を確認してください
4. `marital_status` ごとに喫煙有無(`smoke`)の度数を集計し、`marital_status` ごとの割合を算出してください
5. 婚姻状況が`marital_status == "Married"`の人の喫煙割合を示してください

演習問題1 Rコード

```
# 1. amt_weekends を 0~20本未満・20本以上に階級分け
amt_weekends_cat <- cut(smoking_tbl_df$amt_weekends, breaks = c(-Inf, 20, Inf))

# 2. 度数と相対度数を計算
tab_amt_weekends = table(amt_weekends_cat, useNA="always")
prop.table(tab_amt_weekends)

# 3. amt_weekends と smoke のクロス集計
cross_tab <- table(amt_weekends_cat, smoking_tbl_df$smoke, useNA="always")
cross_tab # 喫煙していない人が欠損している

# 4 婚姻状況による喫煙有無の集計
tab_ex <- table(smoking_tbl_df$marital_status, smoking_tbl_df$smoke)
tab_ex
prop.table(tab_ex, margin = 1)

# 5 table オブジェクトの要素へのアクセス
ptab_ex <- prop.table(tab_ex, margin = 1)
ptab_ex[2,2]
```

Chap2. 外部パッケージを使っの データ集計

- `dplyr` と `janitor` を使ったデータ操作・集計, クロス集計や頻度表の作成
- 組み込み関数を用いた集計と外部パッケージを用いた集計の違い

dplyr と janitor によるデータの集計

dplyr データ操作・集計の基本ツール

janitor クロス集計や頻度表をきれいに出力してくれるツール

大きな特徴: Base Rの方法に比べて追加の処理なく綺麗に集計を表示できる

演習: RStudioで下記のコードを実行してインストール・インポートしてみよう

- コンソールで以下を実行

```
install.packages("dplyr", repos="https://cloud.r-project.org")  
install.packages("janitor", repos="https://cloud.r-project.org")
```

- インストールが完了したら `library(dplyr)` , `library(janitor)` でインポート

dplyr と janitor の標準的な使い方

- Rの組み込みデータセット `iris` を使って標準的な使い方を見てみよう
 - `iris` は初めからRに含まれているので `print(iris)` で内容が確認できる

```
print(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa

がく片の長さ, 幅

花弁の長さ, 幅

種類



[画像の出典 \(R Pubs\)](#)

dplyr と janitor の標準的な使い方

- 単純な集計その1: 一次元離散変数の頻度・相対割合の集計

例 iris データセットの Species 列の集計

集計の目的:

Species 列に含まれるカテゴリ (setosa, versicolor, virginica の3種) ごとに

- 件数 (頻度)
- 全体に占める割合 (% 表示)

を見やすく表示する

dplyr と janitor の標準的な使い方

- 単純な集計その1: 一次元離散変数の頻度・相対割合の集計

例 iris データセットの Species 列の集計

- サンプルコード

```
# 例:iris データセットの Species 列(離散変数)
iris %>%
  tabyl(Species) %>%
  adorn_pct_formatting(digits = 1) # 割合を%表示
```

dplyr と janitor の標準的な使い方

- 単純な集計その1: 一次元離散変数の頻度・相対割合の集計

例 iris データセットの Species 列の集計

- サンプルコード

```
# 例: iris データセットの Species 列(離散変数)
```

```
iris %>%
```

```
  tabyl(Species) %>%
```

```
  adorn_pct_formatting(digits = 1) # 割合を%表示
```

iris 内の Species 列の頻度表(件数 (n) と割合 (percent))を作る

集計した percent 列を「小数 -> パーセント表記」に変換

digits = 1 なので 小数点1桁まで表示

dplyr と janitor の標準的な使い方

- 単純な集計その1: 一次元離散変数の頻度・相対割合の集計

例 iris データセットの Species 列の集計

○ サンプルコード

```
# 例: iris データセットの Species 列(離散変数)
```

```
iris %>%
```

「前の処理の結果を次の関数に渡す」という意味

```
  tabyl(Species) %>%
```

iris -> tabyl() -> adorn_pct_formatting()
と処理がつながる

```
  adorn_pct_formatting(digits = 1) # 割合を%表示
```

iris 内の Species 列の頻度表(件数 (n) と割合 (percent))を作る

集計した percent 列を「小数 -> パーセント表記」に変換

digits = 1 なので 小数点1桁まで表示

dplyr と janitor の標準的な使い方

- 単純な集計その1: 一次元離散変数の頻度・相対割合の集計

例 iris データセットの Species 列の集計

- 実行結果(**演習**: RStudioでコードを実行して集計を表示してみよう)

```
> # 例: iris データセットの Species 列 (離散変数)
> iris %>%
+   tabyl(Species) %>%
+   adorn_pct_formatting(digits = 1) # 割合を%表示
```

Species	n	percent
setosa	50	33.3%
versicolor	50	33.3%
virginica	50	33.3%

> |

各種類の割合は33.3%

各種類ごとに50個のデータ

dplyr と janitor の標準的な使い方

- 単純な集計その2: 一次元連続変数の階級分けと頻度の集計

例 iris データセットの Sepal.Length 列を階級に分けて集計

集計の目的:

Sepal.Length(がく片の長さ)という連続値を一定の幅で区切り(階級分け), 各階級にいくつのデータが入っているかを集計して見やすく表示

dplyr と janitor の標準的な使い方

- 単純な集計その2: 一次元連続変数の階級分けと頻度の集計

例 `iris` データセットの `Sepal.Length` 列を階級に分けて集計

- サンプルコード

```
# Sepal.Length を階級に分けて集計
iris %>%
  mutate(Sepal_cat = cut(Sepal.Length, breaks = seq(4, 8, 0.5))) %>%
  tabyl(Sepal_cat) %>%
  adorn_pct_formatting()
```

dplyr と janitor の標準的な使い方

- 単純な集計その2: 一次元連続変数の階級分けと頻度の集計

例 `iris` データセットの `Sepal.Length` 列を階級に分けて集計

- サンプルコード

```
# Sepal.Length を階級に分けて集計
```

```
iris %>%
```

```
  mutate(Sepal_cat = cut(Sepal.Length, breaks = seq(4, 8, 0.5))) %>%
```

```
  tabyl(Sepal_cat) %>%
```

```
  adorn_pct_formatting()
```

→ `cut()` 関数で連続変数 `Sepal.Length` を階級化して, 新しい列 `Sepal_cat` を作成

- `breaks = seq(4, 8, 0.5)` は 4, 4.5, 5.0, ..., 8.0 という0.5間隔の区切り
- `cut()` はこの区切りに基づいて `Sepal.Length` を分類(カテゴリ化)

dplyr と janitor の標準的な使い方

- 単純な集計その2: 一次元連続変数の階級分けと頻度の集計

例 iris データセットの Sepal.Length 列を階級に分けて集計

- 実行結果(演習: RStudioでコードを実行して集計を表示してみよう)

```
> # Sepal.Length を階級に分けて集計
> iris %>%
+   mutate(Sepal_cat = cut(Sepal.Length, breaks = seq(4, 8, 0.5))) %>%
+   tabyl(Sepal_cat) %>%
+   adorn_pct_formatting()
```

Sepal_cat	n	percent
(4,4.5]	5	3.3%
(4.5,5]	27	18.0%
(5,5.5]	27	18.0%
(5.5,6]	30	20.0%
(6,6.5]	31	20.7%
(6.5,7]	18	12.0%
(7,7.5]	6	4.0%
(7.5,8]	6	4.0%

各階級ごとのデータの割合

各階級ごとのデータ数

dplyr と janitor の標準的な使い方

演習: RStudioで `MedDataSets` データセットに対して `dplyr` と `janitor` を使って以下の集計を実行してみよう

1. `MedDataSets` の `gender` 列に含まれるカテゴリ(FemaleとMaleの2種)ごとに件数(頻度)および全体に占める割合(%表示)を集計して表示
2. `MedDataSets` の `age` 列を 10 歳~100歳まで5歳刻みで階級分けし, 各階級ごとに件数(頻度)および全体に占める割合(%表示)を集計して表示

dplyr と janitor の標準的な使い方

サンプルコード

```
# smoking_tbl_df データセットの gender 列の集計
smoking_tbl_df %>%
  tabyl(gender, show_na = TRUE) %>%
  adorn_pct_formatting(digits = 1) # 割合を%表示

# smoking_tbl_df の age を階級に分けて集計
smoking_tbl_df %>%
  mutate(age_cat = cut(age, breaks = seq(10, 100,
5))) %>%
  tabyl(age_cat, show_na=TRUE) %>%
  adorn_pct_formatting()
```

dplyr と janitor の標準的な使い方

- 単純な集計その3: 二次元離散変数のクロス集計

例 `iris` データの `Species` ごとに花のサイズ(仮のカテゴリ変数)を集計
(ここでは既存の変数をカテゴリ変数に変換)

集計の目的:

`iris` データセットを使い、「種別 (`Species`) 」と、「がく片の長さによるカテゴリ (新たなカテゴリ) 」の関係を表形式で分析

dplyr と janitor の標準的な使い方

- 単純な集計その3: 二次元離散変数のクロス集計

例 `iris` データの `Species` ごとに花のサイズ(仮のカテゴリ変数)を集計
(ここでは既存の変数をカテゴリ変数に変換)

- サンプルコード

```
iris %>%  
  mutate(SizeCat = ifelse(Sepal.Length > 5.5, "Large", "Small")) %>%  
  tabyl(Species, SizeCat) %>%  
  adorn_percentages(denominator = "row") %>%  
  adorn_pct_formatting(digits = 1) %>%  
  adorn_ns()
```

dplyr と janitor の標準的な使い方

- 単純な集計その3: 二次元離散変数のクロス集計

例 `iris` データの `Species` ごとに花のサイズ(仮のカテゴリ変数)を集計
(ここでは既存の変数をカテゴリ変数に変換)

- サンプルコード

```
iris %>%  
  mutate(SizeCat = ifelse(Sepal.Length > 5.5, "Large", "Small")) %>%  
  tabyl(Species, SizeCat) %>%  
  adorn_percentages(denominator = "row") %>%  
  adorn_pct_formatting(digits = 1) %>%  
  adorn_ns()
```

→ 連続変数 `Sepal.Length` をカテゴリ化して新しい変数 `SizeCat` を作成

- `Sepal.Length > 5.5` → `"Large"`
- それ以外 → `"Small"`

dplyr と janitor の標準的な使い方

- 単純な集計その3: 二次元離散変数のクロス集計

例 `iris` データの `Species` ごとに花のサイズ(仮のカテゴリ変数)を集計
(ここでは既存の変数をカテゴリ変数に変換)

- サンプルコード

```
iris %>%  
  mutate(SizeCat = ifelse(Sepal.Length > 5.5, "Large", "Small")) %>%  
  tabyl(Species, SizeCat) %>%  
  adorn_percentages(denominator = "row") %>%  
  adorn_pct_formatting(digits = 1) %>%  
  adorn_ns()
```

→ パーセントの横にデータ数(n)を括弧付きで表示

→ `Species`(`setosa`, `versicolor`, `virginica`) × `SizeCat`(`Large`, `Small`) のクロス集計表を作成

→ 各行(`Species`)の中で、`Large` / `Small` の割合を計算

dplyr と janitor の標準的な使い方

- 単純な集計その3: 二次元離散変数のクロス集計

例 `iris` データの `Species` ごとに花のサイズ(仮のカテゴリ変数)を集計
(ここでは既存の変数をカテゴリ変数に変換)

- 実行結果(**演習**: RStudioでコードを実行して集計を表示してみよう)

```
> iris %>%
+ mutate(SizeCat = ifelse(Sepal.Length > 5.5, "Large", "Small")) %>%
+ tabyl(Species, SizeCat) %>%
+ adorn_percentages(denominator = "row") %>%
+ adorn_pct_formatting(digits = 1) %>%
+ adorn_ns()
```

Species	Large	Small
setosa	6.0% (3)	94.0% (47)
versicolor	78.0% (39)	22.0% (11)
virginica	98.0% (49)	2.0% (1)

各階級ごとのデータの割合

各階級ごとのデータ数

janitor における欠損値の取り扱い

MedDataSets データセットで janitor を用いる際の標準的な欠損値処理を見る

janitor における欠損値の取り扱い

MedDataSets データセットで janitor を用いる際の標準的な欠損値処理を見る

1. tabyl() による1変数集計と NA

```
smoking_tbl_df %>%  
  tabyl(amt_weekends)
```

実行結果 →

amt_weekends 列に対する tabyl() の出力

- n : 件数
- percent : NA も含めた全データに対する割合
- valid_percent : NA を除いたデータ内での割合

A tabyl: 25 × 4				
	amt_weekends	n	percent	valid_percent
	<int>	<int>	<dbl>	<dbl>
1	0	6	0.0035481963	0.014251781
2	1	6	0.0035481963	0.014251781
3	2	7	0.0041395624	0.016627078
4	3	6	0.0035481963	0.014251781
5	4	4	0.0023654642	0.009501188
...				
20	35	5	0.0029568303	0.011876485
21	40	15	0.0088704908	0.035629454
22	45	1	0.0005913661	0.002375297
23	50	2	0.0011827321	0.004750594
24	60	2	0.0011827321	0.004750594
25	NA	1270	0.7510348906	NA

janitor における欠損値の取り扱い

MedDataSets データセットで janitor を用いる際の標準的な欠損値処理を見る

2. `show_na = FALSE` で NA を表示しない

```
smoking_tbl_df %>%  
  tabyl(amt_weekends, show_na = FALSE)
```

実行結果 →

A tabyl: 24 × 3

amt_weekends	n	percent
<int>	<int>	<dbl>
0	6	0.014251781
1	6	0.014251781
2	7	0.016627078
3	6	0.014251781
4	4	0.009501188
5	32	0.076009501
...		
35	5	0.011876485
40	15	0.035629454
45	1	0.002375297
50	2	0.004750594
60	2	0.004750594

NA を表示しないだけであり、**欠損が存在しないことを意味するわけではないことに注意**

janitor における欠損値の取り扱い

MedDataSets データセットで janitor を用いる際の標準的な欠損値処理を見る

3. cut で階級分けしてから tabyl で集計

```
smoking_ex <- smoking_tbl_df %>%  
  mutate(  
    amt_weekends_cat = cut(  
      amt_weekends,  
      breaks = c(-Inf, 20, Inf),  
      labels = c("20本以下", "20本より多い"),  
      right = TRUE  
    )  
  )  
smoking_ex %>%  
  tabyl(amt_weekends_cat, show_na = TRUE)
```

janitor における欠損値の取り扱い

MedDataSets データセットで janitor を用いる際の標準的な欠損値処理を見る

3. cut で階級分けしてから tabyl で集計

```
smoking_ex <- smoking_tbl_df %>%  
  mutate(  
    amt_weekends_cat = cut(  
      amt_weekends,  
      breaks = c(-Inf, 20, Inf),  
      labels = c("20本以下", "20本より多い"),  
      right = TRUE  
    )  
  )
```

cut 関数によって条件
(breaks で指定)に基づいて
データセットを階級分け

```
smoking_ex %>%  
  tabyl(amt_weekends_cat, show_na = TRUE)
```

階級分けしたデータセットを
tabyl 関数で集計

janitor における欠損値の取り扱い

MedDataSets データセットで janitor を用いる際の標準的な欠損値処理を見る

3. cut で階級分けしてから tabyl で集計

実行結果

A tabyl: 3 × 4

	amt_weekends_cat	n	percent	valid_percent
	<fct>	<int>	<dbl>	<dbl>
1	20本以下	344	0.20342992	0.8171021
2	20本より多い	77	0.04553519	0.1828979
3	NA	1270	0.75103489	NA

dplyr と janitor の標準的な使い方

演習: RStudioで `MedDataSets` データセットに対して `dplyr` と `janitor` を使って以下の集計を実行してみよう

1. `MedDataSets` の `amt_weekends` 列を「20本以下」と「20本より多い」の2階級に分け、各階級ごとに件数(頻度)および全体に占める割合(%表示)を欠損を考慮して集計して表示
2. `MedDataSets` の `amt_weekends` 列を「20本以下」と「20本より多い」の2階級に分け、`smoke` 列とのクロス集計表を作成

提出課題: 2で作成したクロス集計表をcsvファイルに出力して提出してください

提出方法: KULMSの課題提出機能

提出期限: 5/20(水)23:55

base Rによる集計と `dplyr` / `janitor` による集計の比較

観点	base R	<code>dplyr</code> / <code>janitor</code>
記述スタイル	関数中心、やや複雑	パイプ <code>%>%</code> による直感的な構文
可読性	やや低い（ネストが多い）	高い（パイプで流れが明快）
拡張性	基本的な処理には十分	グループ集計・ラベル整形に強い
表の整形	<code>table()</code> , <code>prop.table()</code>	<code>tabyl()</code> , <code>adorn_*()</code> で柔軟に整形
出力形式	配列や表形式	データフレーム（tibble）